EXECUTIVE SUMMARY

Automated Daily Trading System

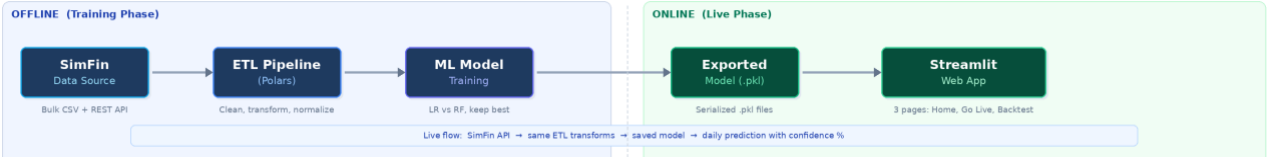
Group 6 | Python for Data Analysis II | March 2026

Live App: greatassignment.streamlit.app **GitHub:** github.com/Lasca/Group_Assignment_Python

1. Project Overview

This project delivers a fully automated system that predicts whether a stock's price will go up or down the next trading day. It collects financial data, processes it into useful signals, trains a prediction model, and presents everything through an interactive web application accessible to anyone online.

System Architecture



5 Tickers	50–59%	3 Pages
MSFT · GOOG · AMZN · TSLA · NVDA	Model Accuracy Range	Home · Go Live · Backtesting

2. Data Sources

All financial data comes from **SimFin**, a platform providing structured stock market data. We used two access methods:

Bulk Downloads were used for training. We downloaded five years of daily share prices (April 2020 to March 2025) for all US-listed companies as the foundation for building our models.

REST API access powers the live application. Through a custom Python wrapper (PySimFin), the web app fetches fresh price data on demand whenever a user requests a prediction.

3. ETL Process

Our ETL pipeline, built with **Polars** (chosen over Pandas for superior performance), prepares raw data for prediction:

Extract	Raw CSV files from SimFin converted to Parquet format for faster reads and efficient storage.
Transform	Fixed date formats, filled missing dividends with zero, used adjusted close prices. Engineered features: daily returns, 5/20-day moving averages, 20-day volatility, 3-day lag returns. Applied min-max normalization.
Load	Clean datasets saved as individual Parquet files per company, ready for model training.

The same transformations are applied during both training and live prediction, preventing data mismatches that could silently degrade quality.

4. Machine Learning Model

The prediction task is binary classification: will the stock close higher or lower tomorrow? We evaluated two algorithms per ticker:

Logistic Regression combines features into a weighted formula, providing a fast, transparent confidence percentage.

Random Forest builds many decision trees and aggregates their votes, capturing more complex non-linear patterns.

For each ticker, we kept whichever model scored higher on unseen test data. The 80/20 train/test split was time-based to avoid information leakage. Accuracies of 50–59% are realistic for stock prediction with basic features; the project's focus is on solid engineering, not maximizing predictions.

5. Web Application

Built with **Streamlit** and deployed on Streamlit Cloud, the application has three pages:

Home Page

Introduces the project with a clean overview, a step-by-step explanation of how the system works (data collection, model training, live predictions), and the full technology stack.

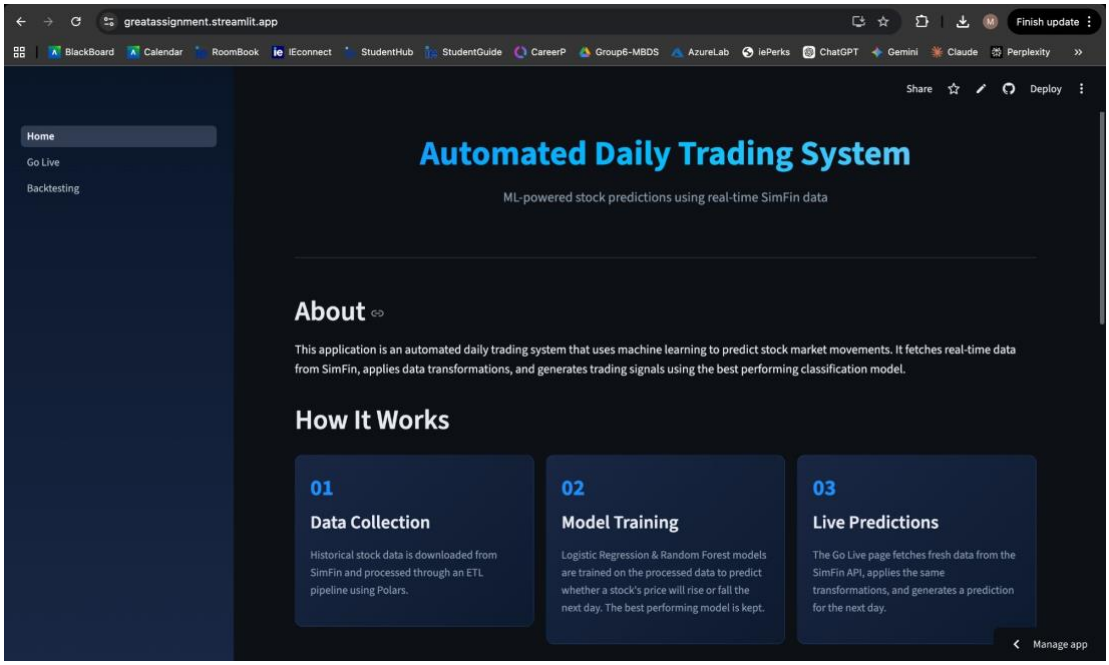


Figure 1: Home page with project overview and How It Works steps

Go Live Page

Users select a ticker and date range. The app fetches live data from the SimFin API, applies the same ETL transformations used during training, and displays the prediction (UP or DOWN) with a confidence percentage. Both raw and transformed data are shown for transparency.

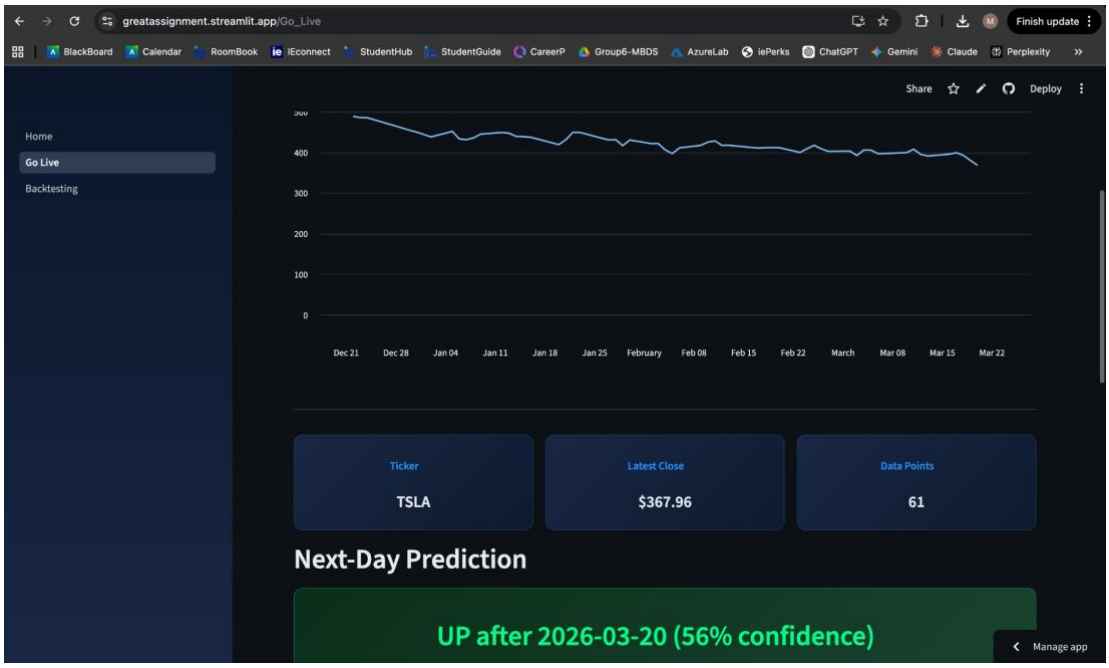


Figure 2: Live prediction for TSLA showing UP with 56% confidence

Backtesting Page

Simulates a Buy-and-Sell trading strategy on historical test data. When the model predicted UP, the strategy buys one share; when it predicted DOWN, it sells one share. Results are compared to a Buy-and-Hold baseline through an interactive portfolio value chart and a complete trade log.

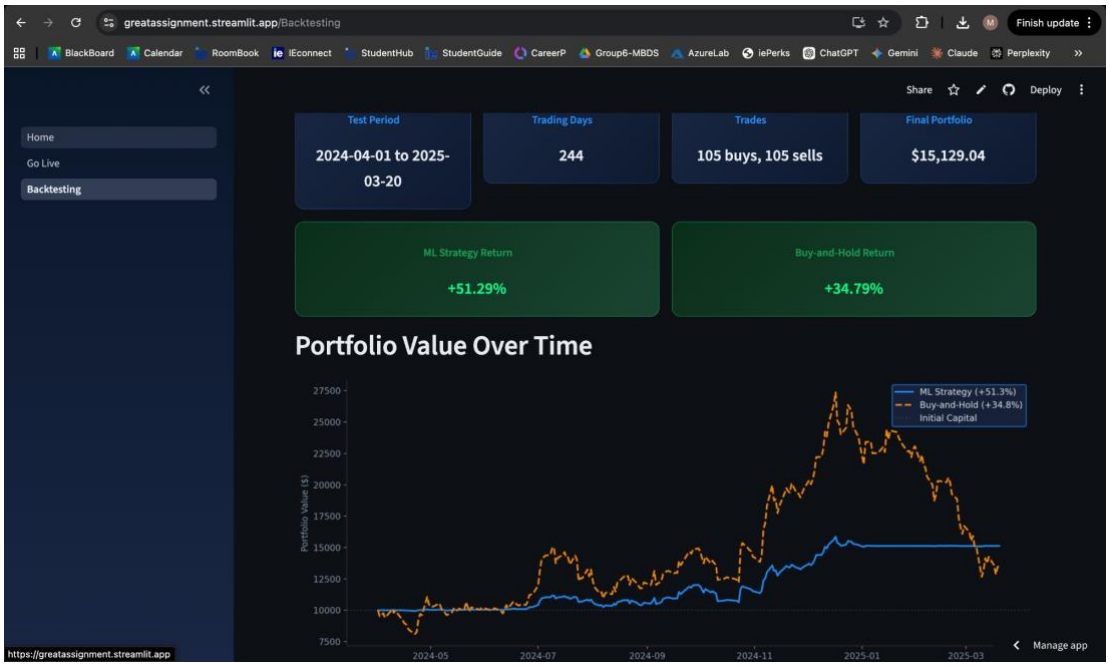


Figure 3: ML Strategy (+51.29%) outperforming Buy-and-Hold (+34.79%)

6. Challenges Faced

Column name mismatches. The SimFin API returns different column names than the bulk download files. This required careful mapping inside the ETL pipeline to ensure both sources produce identical output formats.

Normalization edge cases. Our min-max normalization produced errors when a column had no variation (for example, the Dividend field outside of quarterly payment dates). We added safeguards that detect constant columns and handle them gracefully.

Cloud deployment. Initially, the Parquet data files were excluded from version control, which caused the deployed app to crash on Streamlit Cloud. We resolved this by restructuring the project so that only the small, ML-ready files are committed to the repository, while raw price data is fetched live through the API.

7. Conclusions

This project demonstrates a complete, working data science pipeline: from raw data ingestion, through ETL and model training, to a live web application that anyone can use. While the prediction models achieve modest accuracy (which is expected for stock market forecasting), the engineering behind the system is robust and production-ready.

Key takeaways:

- Polars proved significantly faster than Pandas for our data processing workloads, making it a strong choice for projects dealing with large financial datasets.
- Training multiple models (Logistic Regression and Random Forest) for each ticker and keeping the best performer is a simple but effective strategy.
- Applying identical ETL transformations during training and live inference is critical. Any mismatch between the two pipelines would silently break predictions.
- Cloud deployment introduced practical challenges around secrets management and data file availability that required thoughtful engineering solutions.

Team Members

- Martin Sebastian Schneider Vaquero
- Francisco Javier Santiago Concha Bambach
- Aylin Yasgul
- Qiufeng Cai
- Bader Al Eisa